

Homomorphic Encryption-Enabled Federated Learning for Privacy-Preserving Intrusion Detection in Resource-Constrained IoV Networks

Bui Duc Manh, Chi-Hieu Nguyen, Dinh Thai Hoang and Diep N. Nguyen
School of Electrical and Data Engineering, University of Technology Sydney, Australia.

Abstract—This paper aims to propose a novel framework to address the data privacy issue for Federated Learning (FL)-based Intrusion Detection Systems (IDSs) in Internet-of-Vehicles (IoVs) with limited computational resources. In particular, in conventional FL systems, it is usually assumed that the computing nodes have sufficient computational resources to process the training tasks. However, in practical IoV systems, vehicles usually have limited computational resources to process intensive training tasks, compromising the effectiveness of deploying FL in IDSs. While offloading data from vehicles to the cloud can mitigate this issue, it introduces significant privacy concerns for vehicle users (VUs). To resolve this issue, we first propose a highly-effective framework using homomorphic encryption to secure data that requires offloading to a centralized server for processing. Furthermore, we develop an effective training algorithm tailored to handle the challenges of FL-based systems with encrypted data. This algorithm allows the centralized server to directly compute on quantum-secure encrypted ciphertexts without needing decryption. This approach not only safeguards data privacy during the offloading process from VUs to the centralized server but also enhances the efficiency of utilizing FL for IDSs in IoV systems. Our simulation results show that our proposed approach can achieve a performance that is as close to that of the solution without encryption, with a gap of less than 0.8%.

Index Terms—Intrusion detection, homomorphic encryption, deep learning, IoV.

I. INTRODUCTION

In recent years, the Internet of Vehicles (IoV) has witnessed as a significant research field with the development of intelligent transport systems (ITSs), connected autonomous vehicles (CAV), vehicular ad hoc networks (VANET), and in-vehicle networks (IVNs) [1]. Particularly, IoV is a combination of VANET and IoT, which is built based on the communication standards of VANET [2]. In the IoV network, the vehicles equipped with IoT sensors can transmit and exchange data via the roadside units (RSUs), thereby providing intelligent decision-making based on the collected data. However, the massive network connectivity in IoV leads to substantial concerns in cybersecurity [2]. Regarding cyber threats, the IoV networks are vulnerable to various attacks. For example, the attacker can impersonate one vehicle in the network stream to steal or inject false information into a vulnerable vehicle via a spoofing attack [3]. Additionally, traditional cyberattacks, such as denial of service (DoS), can severely impact IoV networks by overwhelming them with an intensive volume of traffic, thereby disrupting network services. Therefore,

effective detection and defence solutions for cybersecurity in IoV networks are urgently needed to ensure the integrity, reliability, and safety of connected vehicle systems.

Machine Learning (ML) has emerged as a promising approach that can integrate with modern networks to form an intelligent intrusion detection system (IDS). Regarding IoV networks, the authors in [4] propose a Convolutional Long Short Term Memory Network (ConvLSTM) to detect anomalies in IoT sensors integrated CAVs. The simulation results show that the deep learning model can detect various anomalies in sensor data with an F1-score of 97%. Moreover, in [5], the authors evaluate various deep learning techniques to detect attacks in vehicular network traffic, which achieve accuracy from 92% to nearly 99%. However, traditional deep learning approaches operate based on centralized learning paradigms, which may not be efficient for the decentralized nature of IoV/IoT networks. Therefore, Federated Learning (FL) is a significant solution that allows deep learning models to learn with decentralized user-generated data [6]. Regarding IoT networks, the authors in [7] propose a collaborative framework that utilizes FL for intrusion detection in IoT networks. Experiment results using Deep Belief Network (DBN) and Deep Autoencoder (DAE) show the accuracy of detection from 93% to 98%.

Despite the advantage of FL in IDSs, there are still some major challenges when deploying it in practical IoV networks. Specifically, in FL-based IDS in IoV networks, vehicles or RSUs often serve as workers to store and process all the learning tasks (e.g., training and classification) [1]. However, in practice, both RSUs and vehicles usually have limited computing and storage resources, and thus, storing and processing learning tasks at RSUs and VUs are ineffective. It is important to note that in conventional FL processes, a delay from one computing node can cause a delay for the whole system [8]. Therefore, several works propose solutions to upload data from RSUs and vehicles to powerful servers (e.g., centralized servers) for processing [9] [10]. This approach can be very effective in deploying ML algorithms as all the data is collected and processed at the centralized servers. However, it also raises a serious concern regarding the data privacy of VUs, as all the data is now stored and processed externally.

To overcome the above challenges, we propose a novel privacy-preserving FL framework for intrusion detection in IoV networks. The proposed framework can effectively protect

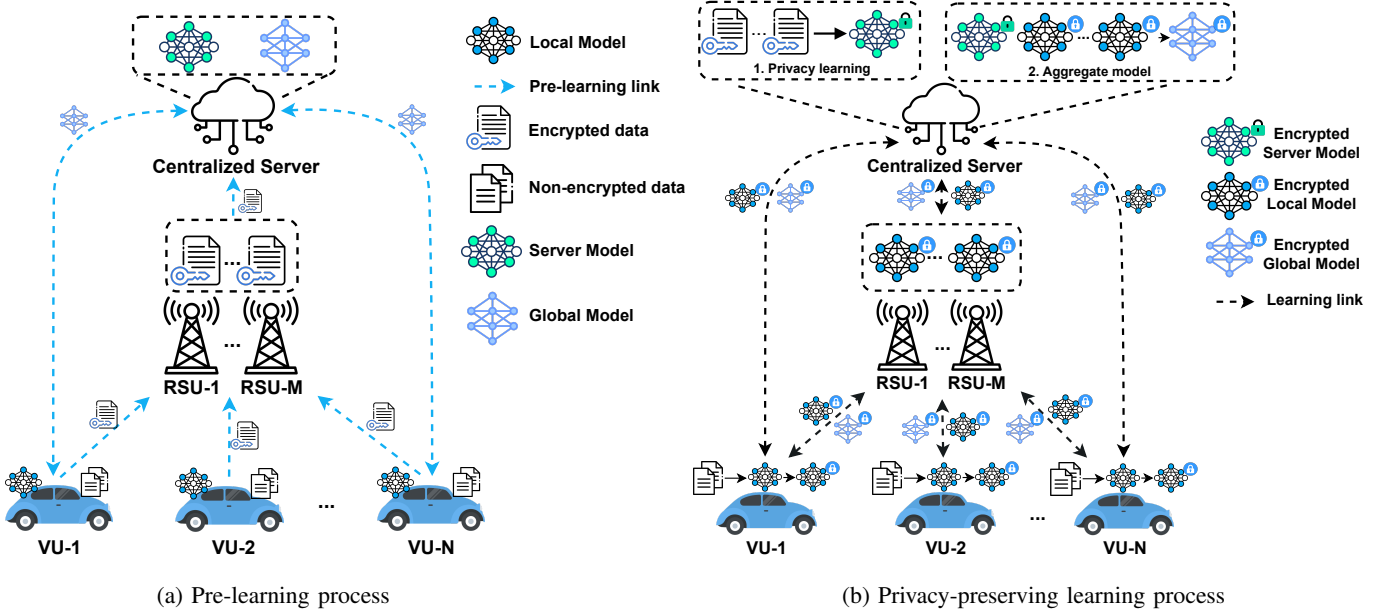


Fig. 1: The proposed privacy-preserving intrusion detection framework including pre-learning and privacy-preserving learning.

VUs' privacy and detect cyberattacks, given VUs' limited computational resources. Specifically, based on the current computing and storage resource capabilities, VUs can decide the amount of data they need to upload to the server for processing. To preserve the privacy of the VUs, this data will be encrypted by employing Homomorphic Encryption (HE) before being uploaded to the server. While this encryption method enhances data privacy, it presents significant challenges for the centralized server, which is tasked with training on the encrypted data offloaded from the VUs. To tackle this issue, we develop a robust training algorithm leveraging the Single Instruction Multiple Data (SIMD) and the bootstrapping capabilities of the underlying HE scheme. This enables direct computation on the quantum-secure encrypted ciphertexts without the need for decryption. This approach not only maintains the confidentiality of data during the offloading process from VUs to the centralized server but also boosts the efficiency of using FL for IDs within IoV networks. Simulation results on real-world datasets show that our proposed framework achieves not only a high accuracy (approximately 91%) in detecting attacks but also exhibits great performance, closely approaching the benchmark without using encryption (with a gap of less than 0.8%).

II. SYSTEM MODEL

The proposed system model is illustrated in Fig. 1. The system consists of a centralized server (CS), M RSUs and N VUs. Initially, the VUs enter the pre-learning phase by assessing their computational resources and determining the optimal amount of data that can be processed locally. The rest of the data will be offloaded to the centralized server for processing. However, before offloading the data to the centralized servers, VUs will generate HE key pairs and use them to encrypt

uploading data. The encrypted data will then be offloaded to the centralized server via RSUs, as illustrated in Fig. 1(a). Upon receiving the offloaded encrypted data, the centralized server will compile it into an encrypted dataset. Using this dataset, two learning models will be developed: the server model and the global model, each serving distinct purposes. The server model will be utilized to train the encrypted dataset within the centralized server, whereas the global model will be distributed to the VUs for local training. Once the global model is sent to the VUs, the privacy-preserving learning process will commence.

The privacy-preserving learning process will be divided into different learning periods. During each learning period, each VU will use the global model to train on its local data. After completing the training, the VU will encrypt its trained model before sending it to the centralized server. Concurrently, the centralized server will train its encrypted data using our proposed CKKS scheme, detailed in Section III-A. Upon receiving all the encrypted trained models from the VUs, the centralized server will aggregate them to create a new global model (the aggregation method is detailed in Section III-B). This updated global model is then sent back to the VUs, and the next learning period begins. This process repeats until the global model converges or until a predefined number of learning periods has been completed.

III. THE PRIVACY-PRESERVING FL FRAMEWORK FOR ENCRYPTED DATA

A. The Classification-based Deep Neural Network for Encrypted Data

To integrate HE with deep neural networks, we propose to use the Cheon-Kim-Kim-Song (CKKS) scheme. The reason is that it allows the encryption and calculation of real numbers,

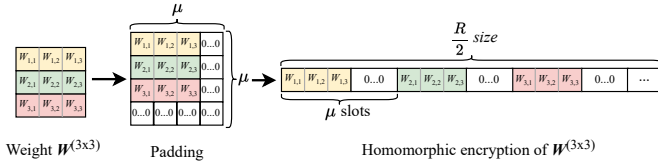


Fig. 2: Encrypt process of a 3x3 matrix W .

which is suitable for deep learning [11]. The CKKS provides basic HE algorithms as follows [12]:

- $SKGen(n)$: generate random secret key s_n^{sk} for user n .
- $PKGen(s_n^{sk})$: create the public key s_n^{pk} for user n based on the secret key s_n^{sk} .
- $Enc(s_n^{pk}, z)$: encrypt vector z into a ciphertext $\hat{z}$ by using the public key s_n^{pk} .
- $Dec(s_n^{sk}, \hat{z})$: decrypt vector $\hat{z}$ into its plain form z by using the secret key s_n^{sk} .
- $Add(\hat{z}, \hat{x})$, $Sub(\hat{z}, \hat{x})$, and $Mul(\hat{z}, \hat{x})$: perform element-wise addition, subtraction and multiplication between two ciphertexts $\hat{z}$ and $\hat{x}$.

Specifically, HE schemes require a ring dimension R , which maintains the security level, multiplication depth, and noise level [13], thereby allowing accurate computations over encrypted data. Following that, to design a deep neural network for encrypted data, we employ the single instruction multiple data (SIMD) from the CKKS scheme, which packs multiple plaintexts into a single ciphertext. The size of ciphertext is denoted as B , where $B = R/2$. Alternatively, the CKKS can encode and encrypt a square matrix of size at most $\mu \times \mu$, where $\mu = \lfloor \sqrt{B} \rfloor$ by initially flattening it into a vector. This thus enables element-wise operation on the plaintext slots concurrently. For clarity, Fig. 2 describes the implementation of the weight matrix encryption method. Let ϕ_i denote the parameter of i linear layer which $\phi_i = (W_i, b_i)$. The weight matrix $W_i^{u \times v}$ with u and v as the input and output dimensions of the layer is applied to the encoding process:

$$Encode(W_i) = Flatten(Pad(W_i, 0, \mu)), \quad (1)$$

where the matrix $W_i^{u \times v}$ is first zero-padded to $W_i^{\mu \times \mu}$ with the size of (μ, μ) to fit within B size. The weight matrix is then flattened to form an encoded vector. After that, this encoded vector is padded to ensure its length equals half of the ring dimension [13]. After that, the encoded vector is encrypted by CKKS, which can be defined by:

$$\hat{W}_i = Enc(s_n^{pk}, Encode(W_i)), \quad (2)$$

where s_n^{pk} is the public key generated by the users. As a result, $\hat{W}_i$ is the encrypted weight of layer i , which can be used to operate with encrypted training data. Therefore, the output of the forward propagation over the i -th layer can be calculated as:

$$\hat{x}_{i+1} = \hat{\sigma}(Add(Mul(\hat{x}_i, \hat{W}_i), \hat{b}_i)), \quad (3)$$

where $\hat{W}_i$ and $\hat{b}_i$ are the encrypted weight and bias at layer i . Particularly, $\hat{\sigma}$ illustrates the polynomial approximation of the

activation function σ using the Chebyshev polynomial [14]. In the considered deep neural network, the *Swish* (SiLU) activation function is chosen due to its advantage in solving the “dying ReLU” problems [15]. Subsequently, the encrypted output vectors consist of the distribution of the classes for classification tasks. It is noted that the *Softmax* function is not applied in this work due to the exponential and inverse functions contained, which are non-homomorphic [12].

In the backpropagation process, we apply the Stochastic Gradient Descent (SGD) for mini-batch regarding the optimization. After calculating the encrypted gradients for each layer, the SGD update of the encrypted weight can be formulated as:

$$\hat{W}_i \leftarrow BootStrap\left(Sub\left(\hat{W}_i, Mul\left(\eta, \frac{\partial \hat{L}}{\partial \hat{W}_i}\right)\right)\right) \quad (4)$$

where $\frac{\partial \hat{L}}{\partial \hat{W}_i}$ is the calculated encrypted gradient of $\hat{W}_i$, which is computed via the derivative of encrypted loss function $\hat{L}$. After the SGD update, the encrypted weight $\hat{W}_i$ is applied to the BootStrap method, which renews the ciphertext, allowing additional computation on $\hat{W}_i$ and reduces the magnitude of accumulated noise [13].

B. The Proposed FL Implementation

In the pre-learning phase, each VU- n evaluates its computing resources and chooses $p_n\%$ of data to offload. After that, they generate a key pair, including a secret key s_n^{sk} and a public key s_n^{pk} . In particular, the VU- n divide its collected dataset $\mathcal{D}_n$ into the local dataset $\mathcal{DR}_n$ and offloaded dataset $\mathcal{DS}_n$ based on effective computing resources of the vehicles. The $\mathcal{DS}_n$ is then encrypted to $\hat{\mathcal{DS}}_n$ to protect the user data. The encrypted data is sent to the CS and combined to form an encrypted dataset $\hat{\mathcal{DS}}$. After that, the CS initializes the non-encrypt global model $\mathcal{M}_g$ and non-encrypt server model $\mathcal{M}_s$, then distributes $\mathcal{M}_g$ to each VU for local training. Subsequently, the public key is used to initialize the encrypted learning model $\hat{\mathcal{M}}_s$ and encrypted global model $\hat{\mathcal{M}}_g$ on the server.

Regarding the privacy-preserving learning phase, we consider T learning rounds. At each round τ , privacy is maintained by the non-encrypted training from the local learning model of VU- n and encrypted training from the privacy-preserving learning model. After finishing the τ^{th} local training round, VU- n encrypts the trained parameters $\hat{\phi}_\tau^n$ and sends them to the CS. The CS retrieves the encrypted parameters from $\hat{\mathcal{M}}_s$ along with the local encrypted parameters and aggregates by the FedAvg algorithm for encrypted data, which can be defined by:

$$\hat{\phi}_g^{(\tau+1)} = Mul\left(\frac{1}{N+1}, Add\left(\sum_{n=1}^N \hat{\phi}_\tau^n, \hat{\phi}_\tau^s\right)\right). \quad (5)$$

The global parameters $\hat{\phi}_g^{(\tau+1)}$ are then updated to the encrypted global model $\hat{\mathcal{M}}_g$ and sent back to the VUs, which is then decrypted by the VUs for the next learning round. The learning process continues until the global model converges and obtains the optimized parameters.

Algorithm 1 Proposed Privacy-Preserving FL Framework

```

1: for  $\forall n \in N$  do
2:   Calculate  $p_n\%$  for offloading
3:   Generate a secret key and a public key:  $s_n^{sk} =$ 
      $SKGen(n)$  and  $s_n^{pk} = PKGen(s_n^{sk})$ 
4:   Split the dataset  $\mathcal{D}_n$  into  $\mathcal{DR}_n$  and  $\mathcal{DS}_n$  where
      $\mathcal{DS}_n = \mathcal{D}_n \times p_n$  and  $\mathcal{DR}_n = \mathcal{D}_n - \mathcal{DS}_n$ 
5:   Generate the encrypted data  $\mathcal{DS}_n = Enc(s_n^{pk}, \mathcal{DS}_n)$ 
6:   Send encrypted data  $\mathcal{DS}_n$  to the CS
7: end for
8: CS combines the received encrypted data  $\mathcal{DS}_n$  into  $\hat{\mathcal{DS}}$ 
9: CS initializes the  $\mathcal{M}_g$  and  $\mathcal{M}_s = \mathcal{M}_g$ 
10: Transmit  $\mathcal{M}_n$  to  $N$  VUs which  $\mathcal{M}_n = \mathcal{M}_g$ 
11: Generate encrypted model  $\hat{\mathcal{M}}_s$  and  $\hat{\mathcal{M}}_g$  via  $s_n^{pk}$  where
      $\phi_n = Dec(\hat{\phi}_n)$ 
12: while  $\tau \leq T_{\max}$  or training process does not converge do
13:    $\hat{\mathcal{M}}_s$  learns the encrypted data  $\hat{\mathcal{DS}}$ 
14:    $\hat{\mathcal{M}}_s$  produces encrypted parameters  $\hat{\phi}_s^\tau$ 
15:   for  $n \in N$  do
16:      $\mathcal{M}_n$  learns the local data  $\mathcal{DR}_n$ 
17:     Calculate local parameters  $\phi_\tau^n$ 
18:     Encrypt local parameters  $\hat{\phi}_\tau^n = Enc(s_n^{pk}, \phi_\tau^n)$ 
19:     Send local encrypted parameters to the CS.
20:   end for
21:   The CS calculates and produces the encrypted
     global model  $\hat{\phi}_g^{(\tau+1)}$ .
22:   Send the updated global model  $\hat{\phi}_g^{(\tau+1)}$  back to  $N$  VUs
23:   for  $\forall n \in N$  do
24:     Decrypt the model  $\phi_g^{(\tau+1)} = Dec(s_n^{sk}, \hat{\phi}_g^{(\tau+1)})$ 
25:   end for
26: end while
27: Predict  $\hat{Y}_n$  based on the encrypted training data  $\hat{X}_n$  at
     each VU- $n$  and optimal global model  $\phi^*$ .

```

In summary, the learning process of the privacy-preserving intrusion detection framework for IoV is described in Algorithm 1.

IV. PERFORMANCE EVALUATION

A. Simulation Setup

In this section, the proposed privacy-preserving model is validated on the real-world dataset of network traffic attacks on IoT devices, named Edge-IIoT dataset [16]. The Edge-IIoT dataset includes 20 million raw normal traffic and attack traffic collected from 13 IoT devices. Attacks can be grouped into the five most common types, including Distributed Denial of Service (DDoS), Injection, Man-in-the-Middle (MitM), Malware, and Reconnaissance. After applying downsample and oversample to overcome the imbalance, the dataset includes 31,400 samples with details presented in Table I. Subsequently, the dataset is divided into training and testing sets (80%-20%). The training and testing sets are then nominalized and scaled within the range of (0,1). Regarding the neural network, we design a fully connected network consisting of an input layer,

TABLE I: The distribution of classes of the dataset

Class	Number of samples
Normal	5,320
DDoS	5,472
MitM	4,000
Injection	5,589
Malware	5,504
Reconnaissance	5,515
Total	31,400

2 hidden layers, and an output layer. The respective layers contain 32, 16, 16, and 6 neurons. Apart from the input layer, each layer is attached to the SiLU activation function.

During the simulation, we assume that the collected dataset of each VU has the same class distribution. Similar to [10], we consider the approach to offload partial data to the server as the benchmark for our proposed framework. Nevertheless, it is noted that in such a benchmark, the FL framework does not consider the privacy of the VUs. In this scenario, we train the non-encrypted data using the non-encrypted model at both CS and VUs. Consequently, the trained global model is employed to evaluate the accuracy of our proposed framework and other benchmarks. In our experiment setup, the proposed framework consists of 2 VUs and 3 VUs, which can send 10% and 20% of their local data.

B. Evaluation Metrics

To evaluate the performance of the detection model, the confusion matrix is utilized, which is suitable for a machine learning-based classification system [17]. We denote TP, TN, FP, and FN as “True Positive”, “True Negative”, “False Positive”, and “False Negative”. Assuming the system consists of C classes, which include normal and attack traffic, the accuracy can be calculated as:

$$Accuracy = \frac{1}{C} \sum_{c=1}^C \frac{TP_c + TN_c}{TP_c + TN_c + FP_c + FN_c}. \quad (6)$$

The macro-average precision and recall are utilized in this term. Given K as the number of classes in the system, the macro-average precision is:

$$Precision = \frac{1}{K} \sum_{k=1}^K \frac{TP_k}{TP_k + FP_k}. \quad (7)$$

The macro-average recall is calculated as follows:

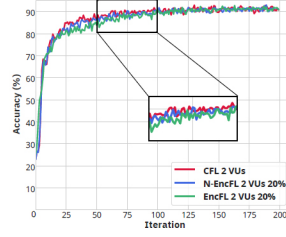
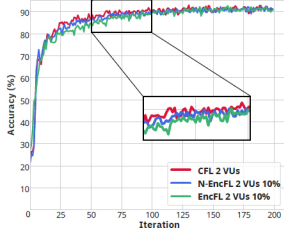
$$Recall = \frac{1}{K} \sum_{k=1}^K \frac{TP_k}{TP_k + FN_k}. \quad (8)$$

C. Simulation Results

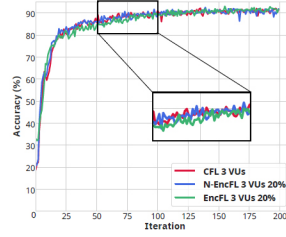
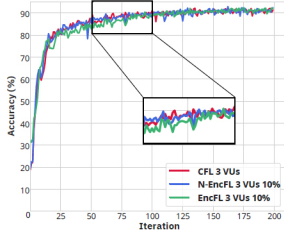
1) *Convergence Analysis*: Fig. 3 illustrates the convergence of learning processes from three approaches, including conventional FL (CFL), FL with non-encrypted offloaded data (N-EncFL) and the proposed privacy-preserving learning (EncFL). As observed in Fig. 3(a) and Fig. 3(b) with 2 VUs, the CFL converges after 70 iterations, while the N-EncFL and EncFL

TABLE II: Simulation results

Model	2 Vehicle Users				3 Vehicle Users			
	N-EncFL		EncFL		N-EncFL		EncFL	
	10% data	20% data	10% data	20% data	10% data	20% data	10% data	20% data
Accuracy	91.728	91.806	91.173	91.142	91.806	91.744	90.926	91.049
Precision	92.767	92.868	92.319	92.254	92.787	92.730	91.992	92.161
Recall	91.875	91.930	91.360	91.322	91.931	91.870	91.118	91.234



(a) 2 VUs with 10% offload data (b) 2 VUs with 20% offload data

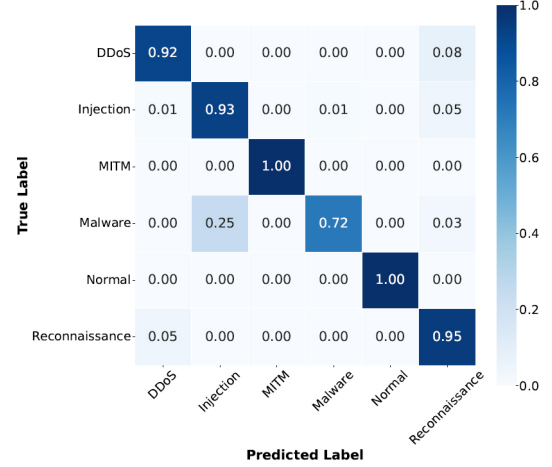


(c) 3 VUs with 10% offload data (d) 3 VUs with 20% offload data

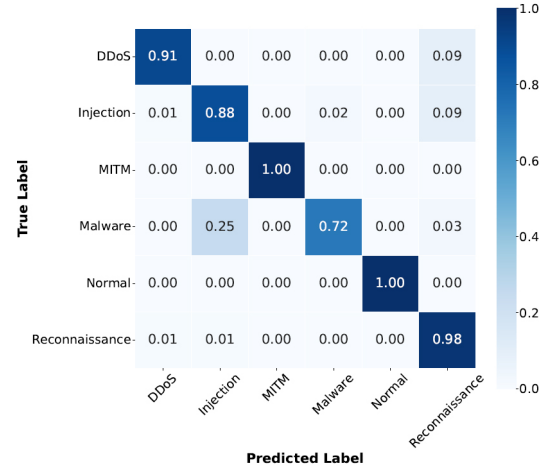
Fig. 3: Convergence of privacy-preserving learning with VUs.

require nearly 100 iterations to reach the convergence. Specifically, due to the different amounts of data handled by the VU, the CFL demonstrates a slightly better convergence rate compared to other approaches. Despite the trivial difference in convergence, the accuracy during the learning process of the three approaches remains nearly identical, stabilizing at approximately 92%. Additionally, Fig. 3(c) and Fig. 3(d) describe the convergences in the scenarios with 3 VUs. Although the N-EncFL and traditional methods converge at nearly the same time, the EncFL require over 100 iterations to reach the convergence, which is slightly longer than other approaches. However, the gap in learning rate, which is about 15 to 20 iterations, is trivial. It is worth noting that the accuracy of EncFL remains consistent with that of the N-EncFL and CFL, regardless of whether the amount of offloaded data is different. As a result, the proposed framework, which operates on encrypted data, achieves the same accuracy as those of the other benchmarks, i.e., N-EncFL and CFL.

2) *Performance Evaluation*: Table II describes the performance in detecting attacks of two and three VUs in the IoV network. Overall, the accuracy, precision and recall of the two scenarios remain nearly identical. Regarding the different amounts of offloaded data, the results for N-EncFL and EncFL are close to those of other methods. Specifically, even when the data sent is 10% or 20%, the N-EncFL with two or three



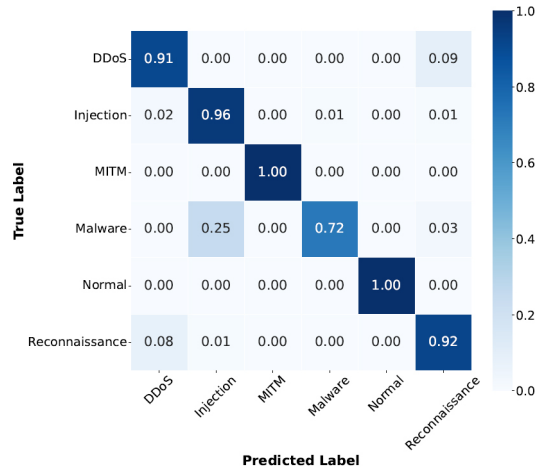
(a) Offloading with 10% non-encrypt data.



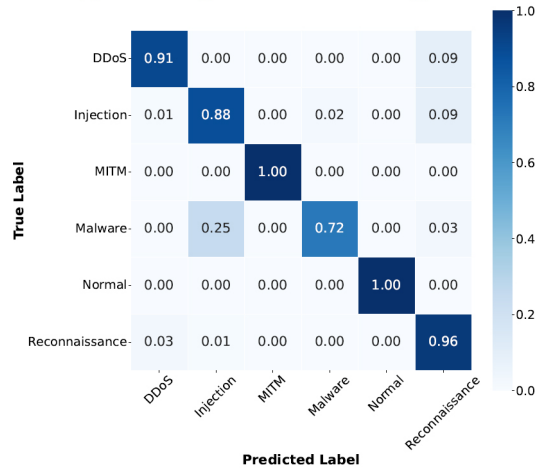
(b) Offloading with 10% encrypted data.

Fig. 4: Classification results of 2 VUs

VUs achieves an accuracy of approximately 91.8%. A similar trend is observed with EncFL, where the accuracy remains consistent regardless of the varying amounts of data offloaded. When comparing the results of EncFL and N-EncFL, we can observe that the accuracy, precision and recall of EncFL are slightly lower than those of the N-EncFL. In detail, the gap between N-EncFL and EncFL with two VUs is from 0.5% to 0.6%. For instance, with 10% offloaded data, EncFL achieves an accuracy of 91.173%, which is 0.55% less than N-EncFL's 91.728%. Additionally, the results of the three VUs show a similar pattern, with EncFL performing 0.6% to 0.8% less than RawFL. However, as observed in Fig. 4 and Fig. 5,



(a) Offloading with 20% non-encrypt data.



(b) Offloading with 20% encrypted data.

Fig. 5: Classification results of 3 VUs

the overall accuracy of 6 classes is nearly the same. The differences primarily lie in the detection of “Injection” and “Reconnaissance”, which accounts for the small gap between N-EncFL and EncFL. Although the accuracy of the “Injection” class of EncFL is less than N-EncFL, EncFL still achieves an 88% detection rate accuracy for the “Injection” attack. As a result, the small gap between N-EncFL and EncFL is acceptable, demonstrating that EncFL can classify each class with a high detection rate.

V. CONCLUSION

In this paper, we have proposed a novel privacy-preserving FL framework for intrusion detection in IoVs with limited computing resources. The proposed framework enables users to offload data to a centralized server, addressing the computational challenges during local training of the vehicles. To ensure user privacy, homomorphic encryption (HE) is applied to the data before offloading it to the server. The encrypted data is then processed by the training algorithm-based HE, which allows the server to learn from the encrypted data without

knowing its content. The proposed framework can protect the privacy of users during the learning process, facilitating the efficient deployment of FL for IDSs in practical IoV networks. The simulation results show that our proposed framework can accurately detect cyberattacks in IoV networks. Although the accuracy of the encrypted neural network is slightly less than that of the raw ones, the gap is acceptable and can be optimized in future works.

REFERENCES

- [1] B. Ji, X. Zhang, S. Mumtaz, C. Han, C. Li, H. Wen, and D. Wang, “Survey on the internet of vehicles: Network architectures and applications,” *IEEE Communications Standards Magazine*, vol. 4, no. 1, pp. 34–41, 2020.
- [2] B. Lampe and W. Meng, “Intrusion detection in the automotive domain: A comprehensive review,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2356–2426, 2023.
- [3] M. Amoozadeh, A. Raghuramu, C.-n. Chuah, D. Ghosal, H. M. Zhang, J. Rowe, and K. Levitt, “Security vulnerabilities of connected vehicle streams and their impact on cooperative driving,” *IEEE Communications Magazine*, vol. 53, no. 6, pp. 126–132, 2015.
- [4] A. Zekry, A. Sayed, M. Moussa, and M. Elhabiby, “Anomaly detection using iot sensor-assisted ConvLSTM models for connected vehicles,” in *IEEE 93rd Vehicular Technology Conference*, 2021, pp. 1–6.
- [5] T. Alladi, V. Kohli, V. Chamola, F. R. Yu, and M. Guizani, “Artificial intelligence (AI)-empowered intrusion detection architecture for the internet of vehicles,” *IEEE Wireless Communications*, vol. 28, no. 3, pp. 144–149, 2021.
- [6] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. Artif. Intell. Stat.*, vol. 54, Apr. 2017, pp. 1273–1282.
- [7] T. V. Khoa, Y. M. Saputra, D. T. Hoang, N. L. Trung, D. Nguyen, N. V. Ha, and E. Dutkiewicz, “Collaborative learning model for cyberattack detection systems in iot industry 4.0,” in *IEEE Wireless Communications and Networking Conference*, 2020, pp. 1–6.
- [8] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, and C. Miao, “Federated learning in mobile edge networks: A comprehensive survey,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 2031–2063, 2020.
- [9] N. Yoshida, T. Nishio, M. Morikura, K. Yamamoto, and R. Yonetani, “Hybrid-fl for wireless networks: Cooperative learning mechanism using non-iid data,” in *IEEE International Conference on Communications*, 2020, pp. 1–7.
- [10] Z. Ji, L. Chen, N. Zhao, Y. Chen, G. Wei, and F. R. Yu, “Computation offloading for edge-assisted federated learning,” *IEEE Transactions on Vehicular Technology*, vol. 70, no. 9, pp. 9330–9344, 2021.
- [11] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology-ASIACRYPT*. Switzerland: Springer, Nov. 2017, pp. 409–437.
- [12] C.-H. Nguyen, Y. M. Saputra, D. T. Hoang, D. N. Nguyen, V.-D. Nguyen, Y. Xiao, and E. Dutkiewicz, “Encrypted data caching and learning framework for robust federated learning-based mobile edge computing,” *IEEE/ACM Transactions on Networking*, pp. 1–16, 2024.
- [13] C. Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.
- [14] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No, “Privacy-preserving machine learning with fully homomorphic encryption for deep neural network,” *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.
- [15] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks*, vol. 107, pp. 3–11, 2018.
- [16] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-IIoTset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning,” *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
- [17] T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, June. 2006.